

Data: _____

RACCOLTA PROVE D'ESAME: DESIGN PATTERN E PROGRAMMAZIONE FUNZIONALE

Il documento contiene 5 prove simulate, ciascuna composta da 2 esercizi, per un totale di 10 esercizi.

Nome: _____

Matr.: _____

Prova 1

1. Per il seguente codice:

```
interface Notifica {  
    void invia(String msg);  
}  
class NotificaBase implements Notifica { ... }  
abstract class NotificaDecorator implements Notifica { ... }  
class NotificaEmail extends NotificaDecorator { ... }  
class NotificaSMS extends NotificaDecorator { ... }
```

1. indicare il design pattern ed i nomi dei ruoli di ciascuna interfaccia e classe fornita,
 2. implementare il codice appropriato di una classe **Client** che chiama i metodi per inviare una notifica decorata sia con Email che con SMS,
 3. disegnare il diagramma UML delle classi fornite e della classe **Client** (indicando nomi classi, nomi metodi, e relazioni fra classi),
 4. disegnare il diagramma UML di sequenza in cui l'esecuzione inizia dalla classe **Client**.
2. Usando la programmazione funzionale di Java, data una lista **flotta** di istanze di **Veicolo** (codice sotto), implementare una classe con:

```
public record Veicolo(String targa, String marca, int anno, int chilometri) { }
```

1. un metodo che trova nella lista **flotta** i veicoli prodotti dopo un anno passato in ingresso, e restituisce la lista delle loro targhe,
2. un metodo che conta il numero di marche diverse presenti nella lista e lo restituisce,
3. un metodo che crea una lista di String contenente le marche presenti, senza ripetizioni,
4. un metodo che restituisce il veicolo con il maggior numero di chilometri.

Nome: _____

Matr.: _____

Prova 2

1. Per il seguente codice, facente parte del sottosistema per un design pattern:

```
interface Canale {
    void iscrivi(Utente u);
    void rimuovi(Utente u);
    void notifica();
}
class CanaleNews implements Canale {
    List<Utente> iscritti;
    ...
}
interface Utente { void aggiorna(String notizia); }
class UtenteApp implements Utente { ... }
```

1. completare l'implementazione del pattern aggiungendo i metodi appropriati alla classe **CanaleNews**,
 2. implementare una classe Client che ha a disposizione un **CanaleNews**, vi iscrive due **UtenteApp** e lancia una notifica,
 3. disegnare il diagramma UML delle classi (indicando nomi classi, nomi metodi, e relazioni fra classi) per il codice fornito e quello aggiunto nei punti a, b,
 4. disegnare il diagramma UML di sequenza illustrando l'esecuzione a partire da una classe appropriata.
2. Usando la programmazione funzionale di Java, data una lista **personale** di istanze di **Dipendente**, implementare una classe con:

```
public record Dipendente(String nome, String dipartimento, double stipendio) { }
```

1. un metodo che trova i dipendenti che appartengono a un dipartimento uguale a un parametro in ingresso, e restituisce la lista dei loro nomi,
2. un metodo che restituisce la lista dei dipartimenti, assicurandosi che non ci siano duplicati,
3. un metodo che calcola e restituisce la somma degli stipendi di tutti i dipendenti della lista,
4. un metodo che restituisce il nome del dipendente con lo stipendio più alto.

Nome:

Matr.:

Prova 3

1. Per il seguente codice:

```
interface SensoreNuovo {
    double getTemperaturaCelsius();
}
class SensoreVecchio {
    double getTemperaturaFahrenheit() { ... }
}
```

1. indicare il design pattern da utilizzare per far comunicare i due sistemi, definendone i ruoli,

2. completare l'implementazione del pattern con una classe **SensoreAdapter** che implementi i metodi appropriati (assumendo una generica formula di conversione),
3. implementare una classe **Client** che utilizza la struttura appena creata per leggere una temperatura in Celsius partendo dal sensore vecchio,
4. disegnare il diagramma UML delle classi fornite, di quelle implementate e della classe **Client**,
5. disegnare il diagramma UML di sequenza in cui l'esecuzione inizia dalla classe **Client**.

2. Usando la programmazione funzionale di Java, data una lista **listaPrenotazioni** di istanze di **Prenotazione**, implementare una classe con:

```
public record Prenotazione(String id, String hotel, int numeroStanze, int giorni)
{ }
```

1. un metodo che calcola la somma del **numeroStanze** per un determinato **hotel** passato in input,
2. un metodo che conta il numero di hotel diversi presenti nella lista e lo restituisce,
3. un metodo che restituisce la lista dei **giorni** di tutte le prenotazioni,
4. un metodo che restituisce l'istanza di **Prenotazione** con il valore massimo del campo **giorni**.

Nome:

Matr.:

Prova 4

1. Per il seguente codice, in cui i componenti formano una struttura ad albero:

```
interface Componente {
    int getDimensione();
}
class File implements Componente { ... }
class Cartella implements Componente {
    List<Componente> figli;
    ...
}
```

1. indicare il design pattern utilizzato ed i nomi dei ruoli di ciascuna interfaccia e classe fornita,
2. completare l'implementazione della classe **Cartella** sviluppando il metodo per il calcolo della dimensione totale,
3. implementare una classe **Client** che crea una struttura con una cartella principale contenente due file e una sottocartella, calcolandone la dimensione complessiva,
4. disegnare il diagramma UML delle classi fornite e della classe **Client**,
5. disegnare il diagramma UML di sequenza a partire dall'invocazione del metodo principale del **Client**.

2. Usando la programmazione funzionale di Java, data una lista **operazioni** di istanze di **Transazione**, implementare una classe con:

```
public record Transazione(String idConto, String tipo, double importo) { }  
// nota: 'tipo' puo' assumere i valori "entrata" o "uscita"
```

1. un metodo che restituisce la lista degli **importo** per le transazioni che hanno **tipo** uguale a un parametro passato in ingresso,
2. un metodo che conta il numero di **idConto** diversi presenti nella lista e lo restituisce,
3. un metodo che calcola la somma degli importi delle sole transazioni di tipo "entrata",
4. un metodo che restituisce l'istanza di **Transazione** con l'importo maggiore in assoluto.

Nome:

Matr.:

Prova 5

1. Per il seguente codice relativo alla gestione dei cambi di stato:

```
interface StatoSemaforo {  
    void scatta(Semaforo s);  
}  
class Semaforo {  
    StatoSemaforo stato;  
    ...  
}  
class Verde implements StatoSemaforo { ... }  
class Giallo implements StatoSemaforo { ... }  
class Rosso implements StatoSemaforo { ... }
```

1. completare l'implementazione del pattern sviluppando il metodo `scatta()` nelle classi concrete in modo da simulare il ciclo Verde -> Giallo -> Rosso -> Verde,
 2. implementare una classe `Client` che inizializza il `Semaforo` allo stato `Verde` e invoca per tre volte la transizione di stato,
 3. disegnare il diagramma UML delle classi (indicando nomi classi, nomi metodi, e relazioni fra classi),
 4. disegnare il diagramma UML di sequenza illustrando l'esecuzione delle transizioni a partire dal `Client`.
2. Usando la programmazione funzionale di Java, data una lista `lavori` di istanze di `Progetto`, implementare una classe con:

```
public record Progetto(String codice, String responsabile, int budget, int  
durataMesi) { }
```

1. un metodo che trova i progetti aventi un budget maggiore di un valore passato come parametro, restituendo la lista dei loro codici,
2. un metodo che restituisce una lista dei nomi dei **responsabile** senza alcuna ripetizione,
3. un metodo che restituisce la somma dei **budget** di tutte le istanze presenti nella lista,
4. un metodo che restituisce la durata media in mesi dei progetti presenti nella lista.